

Universidad de San Andrés, Buenos Aires, Argentina



I206 - Inferencia y Estimación

Trabajo Práctico II

Teoría de la Información:

Codificación de Huffman

Camiscia, Luca

lcamiscia@udesa.edu.ar

Legajo: 36781

Palavecino, Axel

apalavecino@udesa.edu.ar

Legajo: 36778

Pereyra, Agustín

apereyra@udesa.edu.ar

Legajo: 36631

24 de octubre de 2025

Abstract

En este presente trabajo se aborda experimentalmente este principio, investigando cómo se desempeña en la práctica el algoritmo de **Codificación de Huffman**, donde se investiga cómo la capacidad predictiva de una fuente determina su potencial compresión, esto se hace mediante el análisis de textos con propiedades de ocurrencia de caracteres radicalmente diferentes (lenguaje natural, datos estructurados y secuencias genómicas), además de demostrarse empíricamente la optimalidad de este método. Los resultados cuantifican la eficiencia del algoritmo, que minimiza la longitud de codificación promedio ... (**Aca van el resto de los resultados**)

1. Introducción

En la era del **Big Data**, obtener capacidad de representar información de manera concisa y compacta es un desafío fundamental. La *Teoría de la Información*, la cual comenzó **Claude Shannon**,

postula que la compresibilidad de cualquier dato está intrínsecamente limitada por su contenido de incertidumbre, esta medida es cuantificable, y es conocida como *entropía*.

2. Desarrollo Experimental

2.1. Estimación del Modelo probabilístico de la Fuente

El primer paso en la implementación de la **codificación de Huffman** es estimar el modelo probabilístico de la fuente (texto). Esto se logra mediante el análisis de la frecuencia de aparición de cada símbolo en el conjunto de datos. La probabilidad estimada del i -ésimo símbolo s_i con su frecuencia asociada f_i se calcula como:

$$p(x_i) = \frac{f_i}{M} \quad (1)$$

donde cada caracter es una de las N realizaciones totales de la variable aleatoria fuente S y $M = \sum_{j=1}^N f_j$, en otras palabras la longitud total de la fuente. El objetivo en esta etapa es estimar una **función de masa de probabilidad (PMF)** de cada fuente de texto. El resultado de este paso es un conjunto de pares (s_i, p_i) que representan los símbolos y sus probabilidades de ocurrencia, es decir, su **probabilidad empírica** de aparición $p(x)$. Este diccionario constituye el modelo estadístico de la fuente, y es el insumo fundamental sobre el cual operará el algoritmo de **Huffman**.

Es de interés aclarar que en este informe no se realiza un distinción entre mayúsculas y minúsculas, ya que el objetivo principal es reducir el tamaño del texto manteniendo su compresibilidad. La información que aporta una diferenciación entre mayúscula y minúscula es muy pequeña al lado de la complejidad que le aporta a la codificación de Hoffman. La ausencia de mayúsculas al inicio de las oraciones no afecta la comprensión, por lo que no se considera prescindible. No obstante, posteriormente podría aplicarse un algoritmo que reemplace automática-

mente por mayúsculas las letras que sigan a un punto (“.”).

Las características del resultado de la Ecuación (1) se basan en los **Axiomas de la Probabilidad**, es decir, la suma de los p_i es igual a 1, $\sum_{i=1}^N p_i = 1$, la probabilidad de un evento puntual es mayor a cero, $\mathbb{P}(p_i) \geq 0$, y si $A \cap B = \emptyset \Rightarrow \mathbb{P}(A \cup B) = \mathbb{P}(A) + \mathbb{P}(B)$

2.2. Codificación de Huffman

La característica que diferencia este código de otros universales como el *código Morse* es que ninguna cadena de código asignada a algún símbolo es prefijo de otra. A esta propiedad se la conoce como *prefix-free* y es gracias a esto que su decodificación es inmediata.

El código se construye como un árbol binario, cuyas hojas representan un símbolo y a cada rama un elemento binario. Los caracteres de menor frecuencia se encuentran en los mayores niveles de profundidad del árbol, lo que tiene sentido porque lo más frecuentes serán los más rápidos de decodificar y esto es justamente lo que buscamos.

2.3. Construcción del Árbol de Huffman

El árbol es generado de la forma que es representado en el Algoritmo 1, se añaden los símbolos de menor a mayor frecuencia en el árbol creando subárboles de manera que la frecuencia del nodo padre es la suma de la frecuencia de sus hijos. De esta manera, iterativamente se van uniendo los subárboles dejando una estructura como la de la Figura 1

Algoritmo 1 HuffmanTree

Input: pmf $p(s)$ de los caracteres s_i de la cadena S
 $Q \leftarrow$ priority-queue de nodos $n_i = \langle s_i, p(s_i) \rangle$
 # orden ascendente de p
while $\text{len}(Q) > 1$ **do**
 $n_1, p_1 \leftarrow Q.\text{pop}()$
 $n_2, p_2 \leftarrow Q.\text{pop}()$
 $n_{\text{padre}} \leftarrow \langle \text{sub-árbol}(n_1, n_2), p_1 + p_2 \rangle$
 $Q.\text{enqueue}(n_{\text{padre}})$
end while
 $\text{root} \leftarrow Q.\text{pop}()$
return root

Luego la codificación se logra asignando a cada rama un valor binario y para cada símbolo se toma los valores de las ramas del camino único que se realiza desde la raíz hasta la hoja que representa a ese símbolo. En el ejemplo de la figura la codificación para una cadena “abcdabcaba” quedaría tal que $a = 0$, $b = 10$, $c = 110$ y finalmente $d = 111$.

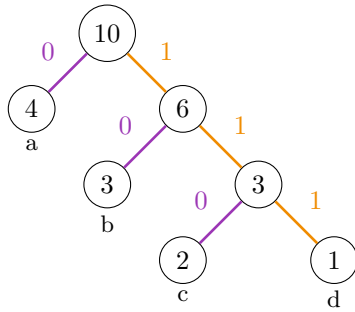


Figura 1: Árbol de Huffman para la cadena $S = \text{“abcdabcaba”}$

2.4. Medidas de Desempeño y Análisis Cuantitativo

Dada una cadena $\mathbf{X}$ de N caracteres x_i únicos y la distribución de probabilidad de los caracteres $p(x_i)$ se realiza la codificación de Huffman para $\mathbf{X}$, de la cual podemos obtener la longitud del código binario para cada símbolo, $\ell(x_i)$. Es de estos datos sencillos de obtener de donde, a partir de ciertos cálculos, es posible estudiar cuantitativamente y procesar la información de cualquier cadena. A continuación algunas descripciones de estas medidas:

Longitud Promedio de Codificación

Es definida como la esperanza de la longitud del código de la cadena y su cálculo es simplemente el costo promedio en bits por símbolo, es decir representa la longitud media del código de cada caracter.

$$L(\mathbf{X}) = \mathbb{E}[\ell(\mathbf{X})] = \sum_{i=1}^N p(x_i) \ell(x_i) \quad (2)$$

Con este podemos darnos una idea de qué tan complejo es la codificación que nos aporta el algoritmo de Huffman. A mayor L , podemos inferir 2 posibles casos, que haya una gran cantidad de caracteres y por lo tanto el árbol binario es de una mayor profundidad o simplemente que la codificación sea ineficiente en términos de compresión.

Longitud Uniforme Mínima de Codificación

Este expresa la mínima cantidad de bits por símbolo necesaria para una codificación uniforme, es decir que no tiene en cuenta la frecuencia de aparición de cada uno de los símbolos.

$$L_{\text{unif}}(\mathbf{X}) = \lceil \log_2(N) \rceil \quad (3)$$

Es muy útil a la hora de evaluar la efectividad de la codificación de Huffman de manera que si $L < L_{\text{unif}}$, ciertamente el algoritmo explota la redundancia de la cadena.

Reducción de Memoria

Con las dos definiciones anteriores, el porcentaje relativo de ahorro de memoria que aporta la codificación se puede estimar mediante la siguiente ecuación

$$R(\mathbf{X}) = \left(1 - \frac{L(\mathbf{X})}{L_{\text{unif}}(\mathbf{X})} \right) \times 100 \% \quad (4)$$

Entropía de Shannon

Es de interés determinar la incertidumbre de una cadena a partir de la información esperada que es necesaria almacenar por cada caracter.

$$H(\mathbf{X}) = - \sum_{i=1}^N p(x_i) \log_2(p(x_i)) \quad (5)$$

En otras palabras, está definida como la $\mathbb{E}[I(x_i)]$ donde $I(x_i)$ es la información en bits del i -ésimo símbolo.

Una forma de interpretar este valor es usando su relación con la longitud promedio de codificación, dado que este está acotado por el primero:

$$H(\mathbf{X}) \leq L(\mathbf{X}) < H(\mathbf{X}) + 1 \quad (6)$$

en caso de que sean iguales, la codificación tiene una eficiencia máxima, y en el caso contrario en que $L > H$ se confirma que existe cierta redundancia en la cadena.

Entropía de Shannon Normalizada

El problema de la medida anterior es que se encuentra escalada respecto a la cantidad de símbolos únicos (N) de la cadena, es por eso que a la hora

de realizar comparaciones entre cadenas de diferentes N es necesario utilizar la versión normalizada (Ecuación (7)).

$$\eta(\mathbf{X}) = \frac{H(S)}{\log_2(N)} \quad (7)$$

donde $\eta \in [0, 1]$ tal que en sus valores extremos $\eta = 1$ evidencia una incertidumbre total o una distribución totalmente uniforme de los símbolos, mientras que en $\eta = 0$ la cadena sería una repetición continua de un solo caracter.

3. Análisis de Datos

Para poder darle una noción numérica a todo lo conceptualizado en este trabajo se hizo uso de tres archivos de texto de formatos diferentes, de manera que se pueda notar una diferencia significativa a la hora de aplicar la teoría en su comparación. En un breve resumen, *adn.txt* es una secuencia de nucleótidos representada solo por 4 caracteres, *mitología.txt* es una descripción y relato sobre la mitología y por último, *tabla.txt* es un archivo en formato .json con datos en su mayoría flotantes y otros como fechas.

El primer paso que debemos realizar es calcular la PMF (Probability Mass Function) de los caracteres, de la que ya mencionamos que parten el resto de medidas utilizadas para la sección. En la Figura 2 se encuentra la representación gráfica de esta función para los textos de ejemplo.

En un primer vistazo de este gráfico, vemos que el archivo *adn.txt* muestra una distribución completamente uniforme entre los cuatro símbolos que lo componen, es decir, $L = L_{\text{unif}}$ que se puede observar efectivamente más adelante. Cada uno aparece con una frecuencia alrededor del 25% lo cual es coherente con la estructura de secuencia del ADN. Esto es un ejemplo sencillo para pensar la información que transmite cada caracter. Partiendo de que, como definimos anteriormente, la información $\sim 1/p$ entonces cada caracter estaría representando $-\log_2(p) \approx 2$ bits de información. Si eliminamos un caracter de la cadena, estaríamos perdiendo demasiada información, y de hecho si hablamos de información genética, quitar un tipo de nucleótido dejaría a esta codificación genética biológicamente inservible.

Lo último se ve reflejado en la compresión relativa en memoria que genera la codificación de Huffman para este texto, la cual es exactamente nula (observable en el Cuadro 1). La cadena no contiene redundancia alguna y por lo que el algoritmo no tiene oportunidad para aprovechar a ahorrar memoria, cada uno de los bits almacenados son indispensables.

Algo fácil de relacionar entre el gráfico y la tabla es la entropía normalizada. A mayor cercanía al 1, significaría una distribución más uniforme en la

frecuencia cada caracter y en nuestro caso con una precisión de 4 decimales sabemos que es casi perfectamente uniforme. En el resto de textos al tener un mayor N y una concentración de ciertos caracteres, este valor es menor.

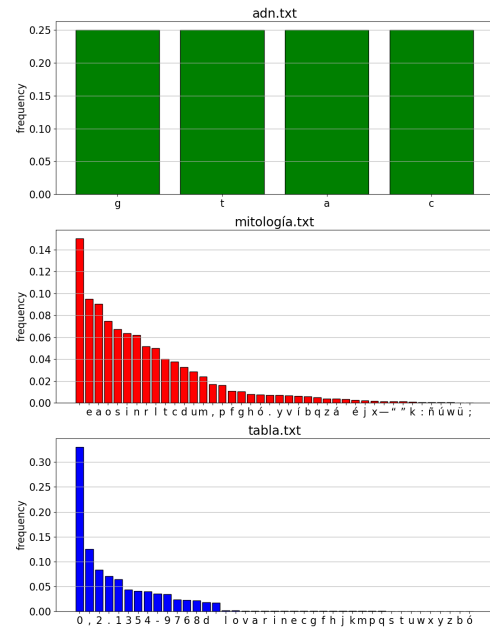


Figura 2: Función de masa de probabilidad de los caracteres de cada uno de los textos de ejemplo

Por otro lado, el archivo *mitologia.txt* muestra una distribución de frecuencias muchos mas variadas. Notamos como unas pocas letras concentran una gran cantidad de las apariciones. Esto podria deberse a la estructura del lenguaje español. Asimismo, vemos como el caracter mas usado es el “_” (espacio) lo que concuerda con la regla gramatical de separar las palabras con este simbolo. (Capaz puede decir algo mas)

Finalmente, en el ultimo archivo al ser una tabla de valores observamos como predominan los caracteres numericos y signos de puntuacion. A diferencia del caso anterior, esta distribución no sigue parametros linguisticos sino de organizacion tipica de informacion numerica.

	Entropía	Entropía Normalizada	Símbolos Únicos	Longitud Promedio	Longitud Uniforme Mínima	Reducción de Memoria
	H [bits/símbolo]	$\eta \in [0, 1]$	N [cantidad]	L [bits/símbolo]	L_{unif} [bits/símbolo]	R [%]
adn.txt	1.9999	1.0000	4	2.0000	2.0000	0.0000
mitología.txt	4.2685	0.7916	42	4.3013	6.0000	28.3112
tabla.txt	3.4668	0.6471	41	3.5161	6.0000	41.3978

Cuadro 1: Resumen de los cálculos de las medidas mencionadas en la sección anterior realizados para los textos de ejemplo

En conjunto, estas diferencias exhiben como las distribuciones de frecuencia están fuertemente determinados por la naturaleza de la información

Los tres textos presentan comportamientos muy distintos, lo que permite observar como la estructura estadística mencionada anteriormente influye directamente en su compresibilidad

El archivo `adn.txt` representa el extremo más simple. Su distribución uniforme produce una entropía baja de 1.99 bit/símbolo (1 normalizada) y una longitud promedio de Huffman igual a la mínima posible (2 bits/símbolo). Ante la inexistencia de diferencias en las probabilidades, el código no puede optimizarse, por lo que resulta en una reducción de memoria nula. Este es el peor caso posible donde la codificación de Huffman no ofrece ninguna ventaja.

Por otra parte, la distribución variada de mi-

tolología.txt típica del lenguaje natural genera una entropía intermedia 4.26 bits/símbolo (0.79 normalizada), y permite al algoritmo asignar códigos más cortos a los símbolos frecuentes. Gracias a esto, se obtiene una longitud promedio de 4.27 bits/símbolo y una reducción de memoria de al 28.31 %. En este caso se puede apreciar claramente como la redundancia del lenguaje natural puede aprovecharse para comprimir información de manera eficiente.

Por su parte, `tabla.txt` es aquel en el que se logra una mayor compresibilidad siendo de 41.30 %. Esto debido a que su frecuencia está dominada por unos pocos caracteres numéricos y signos de puntuación. Esta fuerte concentración reduce la entropía a solo 3.46 bits/símbolo (0.64 normalizada) y permitiendo obtener la mayor eficiencia de codificación.

4. Conclusión